JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.logging

Class Handler

java.lang.Object
    java.util.logging.Handler

Direct Known Subclasses:

MemoryHandler, StreamHandler

```
public abstract class Handler
extends Object
```

A `Handler` object takes log messages from a `Logger` and exports them. It might for example, write them to a console or write them to a file, or send them to a network logging service, or forward them to an OS log, or whatever.

A `Handler` can be disabled by doing a `setLevel(Level.OFF)` and can be re-enabled by doing a `setLevel` with an appropriate level.

`Handler` classes typically use `LogManager` properties to set default values for the `Handler`'s `Filter`, `Formatter`, and `Level`. See the specific documentation for each concrete `Handler` class.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**Handler**()	Default constructor.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
abstract void	close()	Close the Handler and free all associated resources.
abstract void	flush()	Flush any buffered output.
String	getEncoding()	Return the character encoding for this Handler.
ErrorManager	getErrorManager()	Retrieves the ErrorManager for this Handler.
Filter	getFilter()	Get the current Filter for this Handler.
Formatter	getFormatter()	Return the Formatter for this Handler.
Level	getLevel()	Get the log level specifying which messages will be logged by this Handler.
boolean	isLoggable(LogRecord record)	Check if this Handler would actually log a given LogRecord.

abstract void	**publish**(**LogRecord** record)	Publish a LogRecord.
protected void	**reportError**(**String** msg, **Exception** ex, int code)	Protected convenience method to report an error to this Handler's ErrorManager.
void	**setEncoding**(**String** encoding)	Set the character encoding used by this Handler.
void	**setErrorManager**(**ErrorManager** em)	Define an ErrorManager for this Handler.
void	**setFilter**(**Filter** newFilter)	Set a Filter to control output on this Handler.
void	**setFormatter**(**Formatter** newFormatter)	Set a Formatter.
void	**setLevel**(**Level** newLevel)	Set the log level specifying which message levels will be logged by this Handler.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

Handler

```
protected Handler()
```

Default constructor. The resulting `Handler` has a log level of `Level.ALL`, no `Formatter`, and no `Filter`. A default `ErrorManager` instance is installed as the `ErrorManager`.

Method Detail

publish

```
public abstract void publish(LogRecord record)
```

Publish a `LogRecord`.

The logging request was made initially to a `Logger` object, which initialized the `LogRecord` and forwarded it here.

The `Handler` is responsible for formatting the message, when and if necessary. The formatting should include localization.

Parameters:

`record - description of the log event. A null record is silently ignored and is not published`

flush

```
public abstract void flush()
```

Flush any buffered output.

close

```
public abstract void close()
                    throws SecurityException
```

Close the `Handler` and free all associated resources.

The close method will perform a `flush` and then close the `Handler`. After close has been called this `Handler` should no longer be used. Method calls may either be silently ignored or may throw runtime exceptions.

Throws:

SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").

setFormatter

```
public void setFormatter(Formatter newFormatter)
                  throws SecurityException
```

Set a Formatter. This Formatter will be used to format LogRecords for this Handler.

Some Handlers may not use Formatters, in which case the Formatter will be remembered, but not used.

Parameters:

newFormatter - the Formatter to use (may not be null)

Throws:

SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").

getFormatter

```
public Formatter getFormatter()
```

Return the Formatter for this Handler.

Returns:

the Formatter (may be null).

setEncoding

```
public void setEncoding(String encoding)
                 throws SecurityException,
                        UnsupportedEncodingException
```

Set the character encoding used by this Handler.

The encoding should be set before any LogRecords are written to the Handler.

Parameters:

encoding - The name of a supported character encoding. May be null, to indicate the default platform encoding.

Throws:

SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").

UnsupportedEncodingException - if the named encoding is not supported.

getEncoding

```
public String getEncoding()
```

Return the character encoding for this Handler.

Returns:

The encoding name. May be null, which indicates the default encoding should be used.

setFilter

```
public void setFilter(Filter newFilter)
               throws SecurityException
```

Set a Filter to control output on this Handler.

For each call of publish the Handler will call this Filter (if it is non-null) to check if the LogRecord should be published or discarded.

Parameters:

newFilter - a Filter object (may be null)

Throws:

SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").

getFilter

```
public Filter getFilter()
```

Get the current Filter for this Handler.

Returns:

a Filter object (may be null)

setErrorManager

```
public void setErrorManager(ErrorManager em)
```

Define an ErrorManager for this Handler.

The ErrorManager's "error" method will be invoked if any errors occur while using this Handler.

Parameters:

`em - the new ErrorManager`

Throws:

`SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").`

getErrorManager

```
public ErrorManager getErrorManager()
```

Retrieves the ErrorManager for this Handler.

Returns:

`the ErrorManager for this Handler`

Throws:

`SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").`

reportError

```
protected void reportError(String msg,
                           Exception ex,
                           int code)
```

Protected convenience method to report an error to this Handler's ErrorManager. Note that this method retrieves and uses the ErrorManager without doing a security check. It can therefore be used in environments where the caller may be non-privileged.

Parameters:

`msg - a descriptive string (may be null)`

`ex - an exception (may be null)`

`code - an error code defined in ErrorManager`

setLevel

```
public void setLevel(Level newLevel)
              throws SecurityException
```

Set the log level specifying which message levels will be logged by this `Handler`. Message levels lower than this value will be discarded.

The intention is to allow developers to turn on voluminous logging, but to limit the messages that are sent to certain `Handlers`.

Parameters:

`newLevel - the new value for the log level`

Throws:

`SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").`

getLevel

```
public Level getLevel()
```

Get the log level specifying which messages will be logged by this `Handler`. Message levels lower than this level will be discarded.

Returns:

`the level of messages being logged.`

isLoggable

```
public boolean isLoggable(LogRecord record)
```

Check if this `Handler` would actually log a given `LogRecord`.

This method checks if the `LogRecord` has an appropriate `Level` and whether it satisfies any `Filter`. It also may make other `Handler` specific checks that might prevent a handler from logging the `LogRecord`. It will return false if the `LogRecord` is null.

Parameters:

`record - a LogRecord`

Returns:

```
true if the LogRecord would be logged.
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.